

# Programming SQL Server 2019

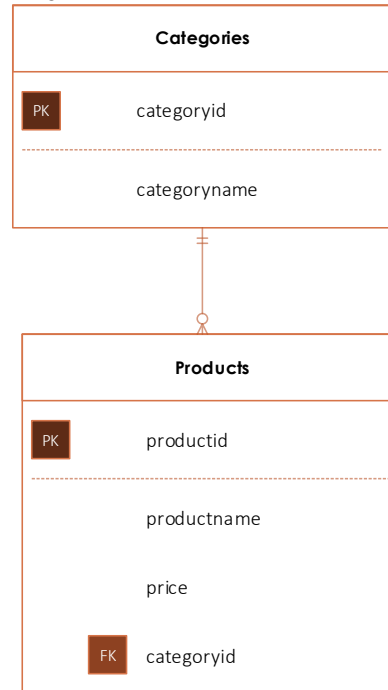
## Database relationship and referential constraint

### Topics

- Database relationships
  - Various types of relationships
  - Implementing relationships
- Foreign Key
  - What is it?
  - Creating relations
  - At column level
  - At table level
- Building database diagram
- Adding foreign key to existing table
- Applying RULES on relation

### Homework

- Create the table with relations as shown in the ER diagram



### Important points

- 
- 
- 

### Summary

- 
- 
-

## Database relationship

- Relationships allow you to describe the connections between different database tables.
- A relationship represents a link between two tables.

## Types of Database Relationships

- There are three different types of database relationships
- Each of these three relationship types exists between two tables.
- One-to-one** relationships occur when each entry in the first table has one, and only one, counterpart in the second table.
- One-to-many** relationships occur when each record in the first table corresponds to one or more records in the second table but each record in the second table corresponds to only one record in the first table.
- Many-to-many** relationships occur when each record in the first table corresponds to one or more records in the second table and each record in the second table corresponds to one or more records in the first table.

## Creating Relationships with Foreign Keys

- You create relationships between tables by specifying a foreign key.
- This key tells the relational database how the tables are related.
- Say we have to store information about Users in our site

|                                                                                                                                                  |                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| name: Bill Gates<br>email: billg@microsoft.com<br>birthday: 28 th Oct 1955<br>gender: Male<br>hobbies: Investments,<br>city: Washington          | name: Steven Jobs<br>email: steve@apple.com<br>birthday: 24 th Feb 1955<br>gender: Male<br>hobbies: creatives, innovations<br>city: San Francisco       |
| name: James Gosling<br>email: james@sun.com<br>birthday: 11 th May 1955<br>gender: Male<br>hobbies: programming, baseball<br>city: San Francisco | name: Srinivas<br>email: sinu@zenbe.com<br>birthday: 27 th Mar 1984<br>gender: Male<br>hobbies: blogging, programming, playing chess<br>city: Bangalore |

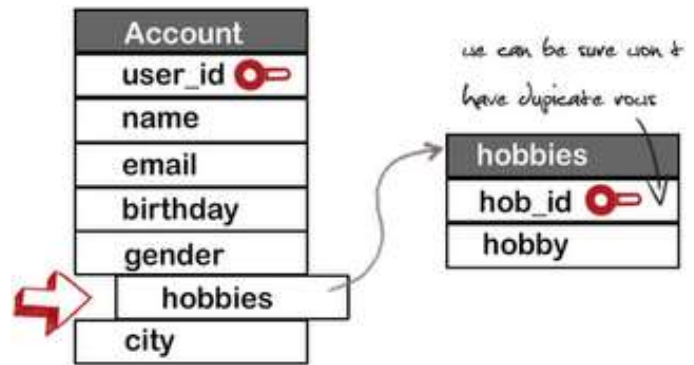
- An easier way to store them in Account table like below

the table name

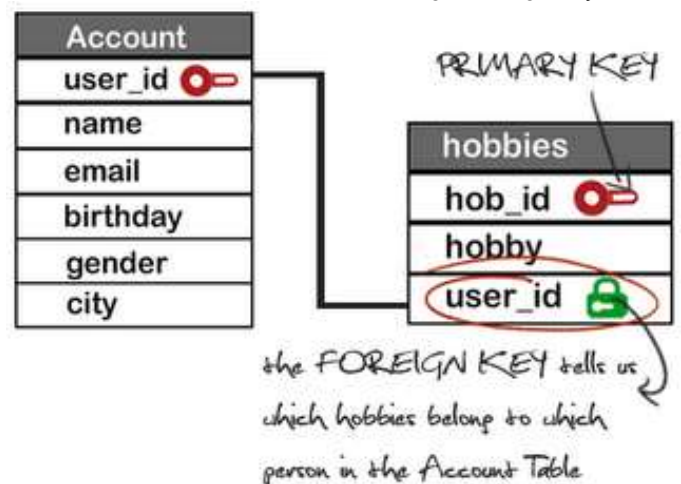
| Account                                                                                     |
|---------------------------------------------------------------------------------------------|
| user_id  |
| name                                                                                        |
| email                                                                                       |
| birthday                                                                                    |
| gender                                                                                      |
| hobbies                                                                                     |
| city                                                                                        |

the key icon means this column is a Primary key

- Every one having different hobbies. We have a better and efficient way.
- Remove the hobbies column and put it in its own table.



- Add name column that will let us identify which hobbies belong to which person in the account table
- The problem we're trying to use name field to somehow let use connect the two tables. But what if two people in the Account table have the same name?
- The solution is to link the tables using a foreign key



## Foreign key

- In relational database, relationship is defined by foreign key
- A foreign key defines child side of a relationship
- A foreign key constraint is often called referential constraint.
- A foreign key is a field in a relational table that matches the primary key column of another table.
- A foreign key is a column that is used to establish and enforce a link between the data in two tables.
- Things to remember:
  - A foreign key can refer to a column in another table in the same database
  - A foreign key can refer a primary key or unique column
  - A foreign key column can allow null
- A foreign key can have a different name that the primary key it comes from.
- The primary key used by a foreign key is also known as a parent key. The table where the primary key is from is known as a parent table.
- The table where foreign key is defined is called child table.
- The foreign key can be used to make sure that the rows in one table have corresponding rows in another table.

- Foreign key values can be null, even though primary key values can't.
- By default, a foreign key applies the following restrictions.
  - In foreign key column, you cannot refer a parent key that does not exist – that is, you cannot create an orphan.
  - You cannot delete a parent key that has any row referencing it in related table.
  - You cannot update a parent key that has any row referencing it in related table.

### Defining foreign key at column level

- Syntax:  
column-name data-type NULL/NOT NULL  
CONSTRAINT constraint-name  
FOREIGN KEY REFERENCES table-name (column-name)
- CONSTRAINT constraint-name part and FOREIGN KEY words are optional
- You can write  
column-name data-type NULL/NOT NULL  
REFERENCES table-name (column-name)

### Defining foreign key at table level

- Foreign key can be defined at table level  
CONSTRAINT constraint-name  
FOREIGN KEY (column-name)  
REFERENCES table-name (column-name)
- Only CONSTRAINT constraint-name optional here

### Creating foreign key

- Consider, for example a database used by a company to store employee information
- That database might have a table called Employees that contains the following information for each member of the company's staff:
  - EmployeeID
  - Name
  - Phone
  - DepartmentID
- In this example, the employee ID is a uniquely generated integer that is assigned to each employee when they are added to the database.
- The DepartmentID is a job code used to reference the department to which the employee belongs.
- In this scheme, an employee may be in one department, there may other employees in the same department.
- The database might also contain a table called Departments with the following additional information about each department:
  - DepartmentID
  - DepartmentName
- The Employees and Departments are linked using DepartmentID field.
- Each employee record has a DepartmentID that must exist in Departments table
- There is one to many relationships exist – one department is associated with many employees
- Let's create the table and relationship

### Try this

```
CREATE TABLE Depratment
(
    DepartmentID INT IDENTITY PRIMARY KEY,
    DepartmentName VARCHAR (30) NOT NULL
)
GO
```

### CREATE TABLE Employees

```
(
    EmployeeID INT IDENTITY PRIMARY KEY,
    EmployeeName VARCHAR (30) NOT NULL,
    Phone VARCHAR (20) NULL,
    DeptmentID INT NOT NULL
    REFERENCES Depratment (DepartmentID)
)
GO
```

### Add some departments

```
INSERT INTO Depratment (DepartmentName)
VALUES ('Admin')
INSERT INTO Depratment (DepartmentName)
VALUES ('Accounts')
INSERT INTO Depratment (DepartmentName)
VALUES ('IT')
GO
SELECT * FROM Depratment
GO
```

- The output is like below

| DepartmentID | DepartmentName |
|--------------|----------------|
| 1            | Admin          |
| 2            | Accounts       |
| 3            | IT             |

- Let's test default behavior of foreign key constraints

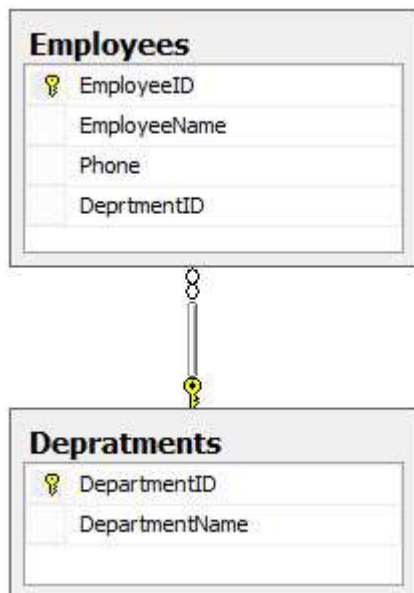
### Try this

```
INSERT Employees (EmployeeName, Phone, DeptmentID)
VALUES ('A. Islam', '98989890', 1)
INSERT Employees (EmployeeName, Phone, DeptmentID)
VALUES ('M. N. Haque', '67676767', 1)
INSERT Employees (EmployeeName, Phone, DeptmentID)
VALUES ('K. Ahmed', '45454545', 2)
GO
```

- The above three work fine
- The following will fail  
INSERT Employees (EmployeeName, Phone, DeptmentID)  
VALUES ('A. Razzak', '00000000', 5)  
GO
- The error message should be like  
The INSERT statement conflicted with the FOREIGN KEY constraint "FK\_Employees\_Deprt\_2A4B4B5E". The conflict occurred in database "sqlstore", table "dbo.Depratment", column 'DepartmentID'. The statement has been terminated.
- By default, a foreign key does not to allow to refer a key that does not exist. There is no department with id 5.

### Building database diagram

- SQL Server allows you to view, create table relations visually
- Visual representations of table are known as Entity Relationship (ER) diagrams.
- Let's create one
  - Expand the database (where you created the tables) in the object explorer
  - Click on Database diagrams
  - A dialog will be shown, click Yes
  - Right click on database diagram and select New Database Diagram
  - Add table dialog will be shown, add Departments and Employees
  - Done
- You will see the relationship as below



## Foreign key UPDATE/DELETE Rules

- Foreign key rules determine the effect of UPDATE or DELETE on a parent key that has associated children
- By default, you cannot update or delete a key that has associated child
- But you can change the rules
- Syntax to add foreign key with rules:  
REFERENCES table-name(column-name)  
ON UPDATE update-rule  
ON DELETE delete-rule
- There are four available rules in SQL Server as follows:
- NO Action:** Not allowed, Error message is generated if a user tries to delete or update a row with data that is involved in a foreign key relationship
- CASCADE-** if a user deletes a row with data that is involved in a foreign key relationship, deletes all the related children. If a user updates a row with data that is involved in a foreign key relationship, all the related children's references is updated.
- SET NULL-** if a user deletes or updates a row with data that is involved in a foreign key relationship, associated values in child table would be set to NULL.
- SET DEFAULT-** if a user deletes or updates a row with data that is involved in a foreign key relationship, associated values in child table would be set to default value.

## Create a Foreign key with rules

Try this

```

CREATE TABLE TSP
(
    tspid INT IDENTITY PRIMARY KEY,
    tspname VARCHAR (30) NOT NULL
)
GO
CREATE TABLE Trainee
(
    traineeid INT IDENTITY PRIMARY KEY,
    traineeName VARCHAR (30) NOT NULL,
    tspid INT REFERENCES TSP (tspid)
    ON DELETE CASCADE
)
GO
INSERT INTO TSP (tspname) VALUES ('Bhuiyan IT')
INSERT INTO TSP (tspname) VALUES ('Creative IT')
INSERT INTO TSP (tspname) VALUES ('US Soft')
GO
  
```

```

INSERT INTO trainee ( traineeName, tspid)
VALUES ('Trainee 1', 1)
INSERT INTO trainee ( traineeName, tspid)
VALUES ('Trainee 2', 3)
INSERT INTO trainee ( traineeName, tspid)
VALUES ('Trainee 3', 3)
GO
SELECT * FROM TSP
SELECT * FROM trainee
GO
  
```

- The output is

| tspid | tspname     |
|-------|-------------|
| 1     | Bhuiyan IT  |
| 2     | Creative IT |
| 3     | US Soft     |

| traineeid | traineeName | tspid |
|-----------|-------------|-------|
| 1         | Trainee 1   | 1     |
| 2         | Trainee 2   | 3     |
| 3         | Trainee 3   | 3     |

- Let's check cascade delete rule

```

DELETE FROM TSP
WHERE tspid =3
GO
SELECT * FROM TSP
SELECT * FROM trainee
GO
  
```

- The output is

| tspid | tspname     |
|-------|-------------|
| 1     | Bhuiyan IT  |
| 2     | Creative IT |
| 3     | US Soft     |

| traineeid | traineeName | tspid |
|-----------|-------------|-------|
| 1         | Trainee 1   | 1     |

- See tsp US Soft with trainee 2 and trainee 3 are deleted